

InvenXis — Sistema de Gestión de Inventarios

Evidencia GA8-220501096-AA1-EV02: Módulos Integrados

Programa	Análisis y Desarrollo de Software (ADSO)
Aprendiz	[NOMBRE DEL APRENDIZ]
Ficha	3235904
Instructor técnico	Carlos Alberto Fuel Tulcán
Fecha de entrega	08 de septiembre de 2026
Repositorio	https://github.com/Zultes-Dev/InvenXis
Versión	1.0

Documento único de entrega: reúne los 5 ítems del instrumento de evaluación (módulos documentados, documento técnico, ambiente, control de versiones, acta de pruebas y aceptación) más manual de usuario y anexos. Se entrega en Zajuna junto con el enlace al repositorio.

Tabla de contenido

- 1. Módulos codificados y documentados (ítem 1)
- 2. Documento técnico del sistema (ítem 2)
- 3. Ambiente de desarrollo y pruebas (ítem 3)
- 4. Código por control de versiones (ítem 4)
- 5. Acta de pruebas y aceptación (ítem 5)
- A. Anexos

1. Módulos codificados y documentados (ítem 1)

Cada módulo indica entradas, salidas y archivos donde vive el código. API base: /api/ con envelope {"success": true, "data": ...} y JWT Bearer.

Autenticación y autorización

Entradas	POST /api/auth/login/, /api/auth/refresh/, /api/auth/logout/, GET /api/auth/me/
Salidas / comportamiento	JWT access+refresh con rotación y blacklist; throttle de login 20/min; RBAC por grupos (Administradores / Operadores).
Código	productos/api_views.py (api_login, api_logout), config/settings.py, frontend/src/contexts/AuthContext.tsx

Productos e inventario

Entradas	CRUD /api/productos/ + filtros (categoría, estado, estado_stock, q) y paginación
Salidas / comportamiento	Lee/escribe Producto (nombre, categoría, precio, cantidad, stock_min, proveedor).
Código	productos/models.py, api_views.py:ProductoListCreateView, frontend/src/pages/ProductosPage.tsx

Proveedores y pedidos

Entradas	CRUD /api/proveedores/, /api/proveedores/{id}/pedidos/, /api/pedidos/{id}/
Salidas / comportamiento	Proveedor con conteos; pedidos con estados pendiente/en_proceso/completado/cancelado.
Código	productos/models.py, frontend/src/pages/ProveedoresPage.tsx

Ventas (punto de venta)

Entradas	GET/POST /api/ventas/ con filtros desde/hasta
Salidas / comportamiento	Crea la venta y descuenta stock en una transacción atómica (select_for_update + F()); sin stock retorna 400 y revierte todo.
Código	productos/serializers.py:VentaCreateSerializer, frontend/src/pages/VentasPage.tsx

Facturación y factura electrónica

Entradas	POST /api/ventas/{id}/facturar/, /api/facturas/, /emitir/, /anular/, /pdf/, /ubl/, /api/notas-credito/
Salidas / comportamiento	Consecutivo atómico FAC/NC, IVA 19%, CUFE SHA-384, UBL 2.1, proveedor DIAN intercambiable (mock incluido); anular crea nota crédito, cancela la venta y devuelve stock.
Código	productos/fe/ (cufe, ubl, proveedores, servicios), productos/models.py:Factura, frontend/src/pages/FacturasPage.tsx

Dashboard y reportes

Entradas	GET /api/dashboard/ (caché 60 s), /api/reportes/inventario mas-vendidos ventas/, exportar excel pdf
Salidas / comportamiento	KPIs, recientes, ranking, exportaciones Excel/PDF.
Código	productos/api_views.py, frontend/src/pages/DashboardPage.tsx, ReportesPage.tsx

Transversales

Entradas	GET /api/health/, /api/docs/ (Swagger), /api/redoc/, /admin/
Salidas / comportamiento	Health-check, documentación OpenAPI, admin Django con 9 modelos, throttling global, caché Redis/local.
Código	config/urls.py, productos/urls_api.py, productos/admin.py

2. Documento técnico del sistema (ítem 2)

2.1 Descripción general. InvenXis es un sistema de gestión de inventarios, proveedores, ventas y facturación electrónica, con arquitectura API-first: backend Django + DRF (lógica y datos) y SPA React (presentación).

2.2 Arquitectura. Petición HTTP → config/urls.py → productos/urls_api.py → APIView/serializers → ORM → PostgreSQL/SQLite → JSON → TanStack Query → React. La presentación vive en la SPA; Django expone API, admin y documentación. Decisiones clave: transacciones atómicas en ventas/facturación, caché de dashboard con invalidación por escritura, throttling, paginación estándar y code-splitting por ruta.

2.3 Modelo de datos. Proveedor 1—N Producto; Proveedor 1—N Pedido 1—N DetallePedido N—1 Producto; Venta 1—N DetalleVenta N—1 Producto; Venta 1—1 Factura 1—N NotaCredito; ContadorDocumento (consecutivos); User/Group de Django para RBAC. Tablas: Proveedor, Producto, Pedido, DetallePedido, Venta, DetalleVenta, ContadorDocumento, Factura, NotaCredito.

2.4 Tecnologías y versiones (verificadas).

Capa	Tecnología y versión	Uso
Lenguaje	Python 3.13.7	Backend
Framework	Django 6.0.7	Servidor y ORM
API	DRF 3.17.1 + SimpleJWT + drf-spectacular	REST, auth, OpenAPI
Base de datos	SQLite (dev) / PostgreSQL 16 (prod)	Persistencia
Caché	Redis 7 (prod) / local (dev)	Dashboard
Servidor prod	Gunicorn + WhiteNoise (Docker)	Despliegue
Lenguaje	TypeScript 6.0.3	Frontend tipado
UI	React 19.2.8 + Vite 8.1.5 + Tailwind v4	SPA
Datos	Axios + TanStack Query 5	HTTP, caché cliente
Calidad	pytest + Playwright + oxlint + tsc	Tests y lint
Runtime	Node 24.16.0 / npm 11.13.0	Frontend

2.5 Instalación y ejecución. Backend: python -m venv venv; pip install -r requirements.txt; copy .env.example .env; python manage.py migrate; python manage.py seed_data; python manage.py runserver (http://127.0.0.1:8000). Frontend: cd frontend; npm install; npm run dev (http://localhost:3000). Demo: admin/admin, operador/operador123. Producción: docker compose up --build.

2.6 Manual de usuario (resumen). Ingresar con usuario/clave → Dashboard (KPIs y recientes) → Productos (buscar, filtrar, crear, editar, eliminar) → Proveedores (directorio y pedidos) → Ventas (registrar con descuento automático de stock) → Facturación (facturar, emitir FE, PDF/UBL, anular con nota crédito — solo Administradores) → Reportes (inventario, más vendidos, ventas, exportar). El escudo del sidebar abre el panel Django admin (solo staff).

2.7 Conclusiones. El sistema cumple el alcance con calidad verificable (45 pruebas backend + 2 e2e en verde); el diseño atómico evita sobregiros de stock y duplicados de factura; queda como trabajo futuro la integración real con la DIAN (certificado y proveedor tecnológico).

3. Ambiente de desarrollo y pruebas (ítem 3)

Herramienta	Versión
Python	3.13.7
Django / DRF	6.0.7 / 3.17.1
Node.js / npm	24.16.0 / 11.13.0
React / Vite / TypeScript	19.2.8 / 8.1.5 / 6.0.3
Base de datos dev	SQLite (db.sqlite3, ignorada por git)
Base de datos prod	PostgreSQL 16 + Redis 7 (docker-compose)
Control de versiones	Git + GitHub
Navegador de pruebas	Chromium (Playwright)
SO de desarrollo	Windows + PowerShell 5.1

Reproducción del ambiente (pasos): 1) python -m venv venv y activar; 2) pip install -r requirements.txt; 3) copiar .env.example → .env; 4) python manage.py migrate && python manage.py seed_data; 5) python manage.py runserver; 6) en otra terminal cd frontend && npm install && npm run dev; 7) verificar con pytest -q y npm run build. Variables clave en .env.example: SECRET_KEY, DEBUG, DB_ENGINE/Nombre, JWT_*, REDIS_URL, CORS_ALLOWED_ORIGINS.

4. Código por control de versiones (ítem 4)

Repositorio remoto (público, accesible al instructor): <https://github.com/Zultes-Dev/InvenXis>, rama principal main. 18 commits con mensajes convencionales (feat/fix/chore/docs/ci). Historial reciente:

```
ee2616f feat: modulo de ventas completo con facturacion y FE DIAN-ready
6ca0e78 fix: login conserva next al admin + RBAC demo
95e020e feat: acceso al panel Django admin desde el sidebar
39e5ced feat: hardening senior x10 (health, throttle, factura uuid, a11y)
cd0dabd feat: seed_data --force purga demo y resiembra determinista
390eb1e feat: TanStack Query en SPA, API en /api, e2e Playwright, retiro SSR
9d21c8c feat: P0+P1 hardening (venta atómica, logout blacklist, docker)
d12bb07 chore: rename InvenSoft Pro to InvenXis
```

CI (GitHub Actions): job backend (pytest --cov), job frontend (npm run build + lint) y job e2e (Django + seed + Vite + Playwright).

5. Acta de pruebas y aceptación (ítem 5)

5.1 Plan de pruebas manual. Casos ejecutados contra el sistema real; se compara resultado esperado vs. obtenido:

ID	Caso	Esperado	Obtenido	Estado
TC-01	Inicio de sesión válido	Acceso y panel principal	Acceso y panel (API + e2e)	Cumple
TC-02	Crear producto	Registro creado	Creado vía API y UI	Cumple
TC-03	Listar/buscar/paginar	Datos correctos y paginados	Paginación y filtros OK	Cumple
TC-04	Editar producto	Dato actualizado	Actualizado vía API y UI	Cumple
TC-05	Eliminar producto	Registro eliminado	Eliminado vía API y UI	Cumple
TC-06	Registrar venta	Venta creada y stock descontado	Descuento atómico exacto	Cumple
TC-07	Venta sin stock	Rechazo 400 sin crear venta	400 + rollback total	Cumple
TC-08	Facturar venta (IVA 19%)	Factura con totales correctos	Subtotal/IVA/total exactos	Cumple
TC-09	Emitir FE (CUFE+UBL)	Validada con CUFE y XML	Aceptada mock, XML válido	Cumple
TC-10	Anular factura + RBAC	NC creada, stock devuelto; operador 403	NC + stock + 403 OK	Cumple
TC-11	Logout blacklist + throttle	Refresh invalidado; abuso 429	Blacklist y 429 OK	Cumple
TC-12	Integración DIAN en producción	Emisión real ante DIAN	Proveedor mock; falta certificado y proveedor tecnológico	No cumple

5.2 Ejecución y resultados. Backend: 45 pruebas pytest en verde (modelos 21 + venta atómica 3 + seed 2 + hardening 6 + RBAC 3 + facturación 10), sin warnings. Frontend: tsc + vite build OK, oxlint 0 errores. E2E Playwright: 2/2 (smoke + facturación punta a punta contra Django real).

5.3 Observaciones y pendientes. TC-12 documentado como pendiente con honestidad técnica: la arquitectura prevé el proveedor real (`productos/fe/proveedores.py:get_proveedor`), pero la emisión productiva exige certificado digital, firma del UBL, rango de numeración autorizado y validación contra la resolución DIAN vigente. Prioridad: media (no bloquea el uso del sistema en modo demostración).

5.4 Acta de aceptación (formato).

Proyecto	InvenXis — Sistema de Gestión de Inventarios (v1.0)
Evidencia	GA8-220501096-AA1-EV02 — Módulos Integrados
Aprendiz / Ficha	[NOMBRE DEL APRENDIZ] / 3235904
Fecha	08 de septiembre de 2026
Resultado	11 casos Cumplen / 1 pendiente documentado (TC-12)
Soporte	Repositorio Git (https://github.com/Zultes-Dev/InvenXis), pruebas automatizadas, manuales

Declaración: el evaluador/cliente, luego de revisar la solución y verificar el cumplimiento de los requisitos acordados, acepta el entregable con el pendiente TC-12 documentado. Se firma en señal de conformidad.

Evaluador / Instructor	Aprendiz
Nombre: Carlos Alberto FUEL TULCÁN Firma: _____ Fecha: 08 de septiembre de 2026	Nombre: [NOMBRE DEL APRENDIZ] Firma: _____ Fecha: 08 de septiembre de 2026

A. Anexos

Endpoints principales (base /api/): auth/login, auth/refresh, auth/logout, auth/me, productos/, proveedores/, proveedores/{id}/pedidos/, pedidos/{id}/, ventas/, ventas/{id}/facturar/, facturas/, facturas/{id}/, facturas/{id}/emitir|anular|pdf|ubl/, notas-credito/, dashboard/, reportes/inventario|mas-vendidos|ventas/, reportes/exportar/excel|pdf/{tipo}/, health/, docs/, redoc/.

Credenciales demo (solo desarrollo): admin/admin (superuser), operador/operador123 (grupo Operadores).

Enlaces: app <http://localhost:3000>, API <http://127.0.0.1:8000>, docs <http://127.0.0.1:8000/api/docs/>, admin <http://127.0.0.1:8000/admin/>, repo <https://github.com/Zultes-Dev/InvenXis>.